MODÜL 6

SQL Orta

GROUP BY • HAVING • Aggregations • JOIN • UNION • Subquery

Amaç: Ham tablolardan iş sorularına yanıt üretebilen, doğru grain'de aggregate alan, tabloları güvenle birleştiren ve sorgu sonucunu doğrulayabilen SQL analisti seviyesine gelmek.

Konu	Bu modülde kazanacağın yetkinlik
GROUP BY / HAVING	Segment, dönem, ürün ve müşteri kırılımında KPI üretmek
COUNT / SUM / AVG	Toplam, adet ve ortalamaları doğru grain'de hesaplamak
JOIN'ler	Birden fazla tablodan analitik dataset üretmek
UNION	Benzer yapıdaki sonuçları satır bazında birleştirmek
Subquery	Ara sonucu başka sorgunun filtresi veya kaynağı olarak kullanmak
Kontrol yaklaşımı	Duplicate, cardinality ve double-counting hatalarını yakalamak

Çıktı: Satış analizi + gerçek SQL case + mülakat hazırlığı

1. SQL Orta Seviyede Ne Değişir?

SQL temel seviyede satır seçmeyi ve filtrelemeyi öğrenirsin. Orta seviyede ise asıl analitik çalışma başlar: satırları gruplarsın, KPI hesaplar, tabloları birleştirir ve sonuçların neden doğru olduğunu kontrol edersin.

Analistin düşünme sırası

- İş sorusunu netleştir: “Aylık kategori geliri nedir?” gibi.
- Analiz grain’ini belirle: satırın neyi temsil ettiğini bil.
- Gerekli tabloları ve join anahtarlarını belirle.
- Filtreleri uygula, sonra aggregate et.
- Sonucu kontrol sorgularıyla doğrula: satır sayısı, distinct key, toplam karşılaştırması.

Grain (tane seviyesi): Bir satırın temsil ettiği en küçük iş birimidir. Örneğin orders tablosunda bir satır bir sipariş; order_items tablosunda bir satır siparişteki bir ürün olabilir. JOIN hatalarının büyük kısmı grain’i bilmemekten çıkar.

Örnek e-ticaret şeması

Tablo	Grain	Anahtar	Önemli alanlar
customers	1 satır = müşteri	customer_id	city, signup_date
orders	1 satır = sipariş	order_id	customer_id, order_date, status
order_items	1 satır = sipariş ürünü	order_id + product_id	quantity, unit_price
products	1 satır = ürün	product_id	category, brand
payments	1 satır = ödeme işlemi	payment_id	order_id, amount, method

2. Aggregate Fonksiyonlar: COUNT, SUM, AVG

Aggregate fonksiyonlar birden fazla satırı özetleyerek tek bir değer üretir. Veri analizinde revenue, order count, AOV, müşteri sayısı gibi KPI'ların çoğu bu fonksiyonlarla oluşturulur.

COUNT çeşitleri

```
SELECT COUNT(*) AS row_count, COUNT(customer_id) AS non_null_customer_count, COUNT(DISTINCT customer_id) AS unique_customers FROM orders;
```

İfade	Ne sayar?	Risk
COUNT(*)	Tüm satırlar	Duplicate satır varsa onları da sayar
COUNT(column)	NULL olmayan değerler	NULL kayıtlar sayılmaz
COUNT(DISTINCT column)	Tekil değerler	Doğru key seçilmezse anlam değişir

SUM ve AVG

```
SELECT SUM(total_amount) AS revenue, AVG(total_amount) AS avg_order_value FROM orders WHERE status = 'completed';
```

AVG tuzağı: "Ortalama müşteri harcaması" ile "ortalama sipariş tutarı" aynı değildir. İlki müşteri grain'inde önce müşteri toplamını hesaplamayı gerektirir; ikincisi sipariş grain'inde doğrudan AVG(total_amount) olabilir.

NULL davranışı

SUM ve AVG çoğu SQL motorunda NULL değerleri hesaplamaya dahil etmez. Bu yüzden NULL'ın "0" anlamına gelip gelmediğini iş bağlamında ayrıca kontrol etmelisin.

3. GROUP BY: KPI’ları Kırılımlara Ayırmak

GROUP BY satırları belirlediğin boyutlara göre gruplar ve her grup için aggregate üretir. Veri analistinin “hangi segment daha iyi?”, “hangi ay büyüdük?” gibi sorularına verdiği temel SQL cevabıdır.

Kategori bazında gelir

```
SELECT p.category, SUM(oi.quantity * oi.unit_price) AS revenue FROM order_items oi JOIN products p ON p.product_id = oi.product_id GROUP BY p.category ORDER BY revenue DESC;
```

Birden fazla boyutla GROUP BY

```
SELECT DATE_TRUNC('month', o.order_date) AS month, p.category, SUM(oi.quantity * oi.unit_price) AS revenue FROM orders o JOIN order_items oi ON oi.order_id = o.order_id JOIN products p ON p.product_id = oi.product_id WHERE o.status = 'completed' GROUP BY DATE_TRUNC('month', o.order_date), p.category ORDER BY month, revenue DESC;
```

Analitik okuma: GROUP BY listesinde hangi alanlar varsa sonuç tablosunun grain’i de büyük ölçüde o alanların kombinasyonudur. Yukarıdaki sonuçta bir satır = ay + kategori.

GROUP BY’da sık hata

SELECT bölümünde aggregate olmayan bir alan gösteriyorsan çoğu SQL motorunda onu GROUP BY’a eklemen gerekir. Aksi hâlde sorgu hata verir veya bazı motorlarda belirsiz sonuç üretebilir.

4. HAVING: Gruplanmış Sonuçları Filtrelemek

WHERE ham satırları aggregate işleminden önce filtreler. HAVING ise GROUP BY ile oluşan grupları aggregate işleminden sonra filtreler.

WHERE ve HAVING farkı

Kriter	WHERE	HAVING
Çalıştığı aşama	GROUP BY'dan önce	GROUP BY'dan sonra
Filtrelediği şey	Ham satırlar	Gruplar / aggregate sonuçlar
Örnek	status = completed	SUM(revenue) > 100000

```
SELECT customer_id, COUNT(*) AS order_count, SUM(total_amount) AS revenue FROM orders WHERE status = 'completed' GROUP BY customer_id HAVING COUNT(*) >= 5 AND SUM(total_amount) >= 10000 ORDER BY revenue DESC;
```

İş sorusu: En az 5 tamamlanmış siparişi ve 10.000 TL üzeri toplam harcaması olan sadık müşteriler kimler? WHERE siparişleri temizler; HAVING müşteri gruplarını seçer.

Yanlış kullanım örneği

```
-- Yanlış / anlamsız yaklaşım WHERE SUM(total_amount) > 10000 -- Doğru HAVING SUM(total_amount) > 10000
```

5. JOIN Mantığı: Tabloları Birleştirmek

Gerçek şirket verisi tek tabloda durmaz. Sipariş ayrı, müşteri ayrı, ürün ayrı tabloda tutulur. JOIN, ortak anahtarlar üzerinden bu tabloları aynı analitik sorguda birleştirir.

Dört temel JOIN türü

JOIN	Ne getirir?	Tipik analitik kullanım
INNER JOIN	Her iki tarafta eşleşen kayıtlar	Sipariş vermiş müşterileri ürünleriyle analiz etmek
LEFT JOIN	Sol tablonun tamamı + sağ eşleşmeler	Hiç sipariş vermemiş müşterileri bulmak
RIGHT JOIN	Sağ tablonun tamamı + sol eşleşmeler	Daha az kullanılır; tablo yönü değiştirilebilir
FULL OUTER JOIN	Her iki tablonun tüm kayıtları	İki sistem arasındaki reconciliation / eşleşme kontrolü

```
SELECT o.order_id, c.customer_id, c.city, o.order_date, o.total_amount FROM orders o JOIN customers c ON c.customer_id = o.customer_id;
```

JOIN anahtarı seçimi: “Kolon isimleri aynı” diye join yapılmaz. İş ilişkisini temsil eden key kullanılır. Yanlış key sessizce satır çoğalmasına ve KPI bozulmasına yol açabilir.

6. INNER JOIN ve LEFT JOIN’i Analitik Kullanmak

INNER JOIN: sadece eşleşenler

```
SELECT c.customer_id, c.city, o.order_id FROM customers c INNER JOIN orders o ON o.customer_id = c.customer_id;
```

Bu sorguda hiç siparişi olmayan müşteriler görünmez. Bu davranış retention/churn veya müşteri tabanı analizi yaparken kritik olabilir.

LEFT JOIN: eşleşmeyenleri de koru

```
SELECT c.customer_id, c.city, COUNT(o.order_id) AS order_count FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id GROUP BY c.customer_id, c.city;
```

Hiç sipariş vermeyen müşteriler

```
SELECT c.customer_id, c.city FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL;
```

LEFT JOIN + WHERE tuzağı: Sağ tablodaki bir koşulu WHERE bölümüne yazarsan NULL kalan satırları eleyip sorguyu fiilen INNER JOIN’e dönüştürebilirsin. Koşulun amacı eşleşmeyi sınırlamaksa çoğu zaman ON içine taşımak gerekir.

7. JOIN Cardinality ve Double-Counting

SQL orta seviyedeki en önemli becerilerden biri JOIN sonrasında satır sayısının neden değiştiğini anlamaktır. Özellikle 1-N ve N-N ilişkilerde aynı iş olayı birden fazla satıra çoğalabilir.

Problem örneği

Bir siparişte 3 ürün ve aynı siparişe ait 2 ödeme işlemi varsa order_items ile payments tablolarını yalnızca order_id üzerinden aynı anda join etmek 3 × 2 = 6 satır üretebilir. Bu durumda ödeme toplamını SUM etmek 2 veya 3 kat fazla sonuç verebilir.

```
-- Riskli sorgu SELECT SUM(p.amount) AS payment_total FROM orders o JOIN order_items oi ON
oi.order_id = o.order_id JOIN payments p ON p.order_id = o.order_id;
```

Daha güvenli yaklaşım: önce aggregate, sonra join

```
WITH payment_by_order AS ( SELECT order_id, SUM(amount) AS payment_amount FROM payments GROUP BY
order_id ), item_by_order AS ( SELECT order_id, SUM(quantity * unit_price) AS item_revenue FROM
order_items GROUP BY order_id ) SELECT o.order_id, p.payment_amount, i.item_revenue FROM orders o
LEFT JOIN payment_by_order p ON p.order_id = o.order_id LEFT JOIN item_by_order i ON i.order_id =
o.order_id;
```

Kontrol alışkanlığı: JOIN’den önce ve sonra COUNT(*), COUNT(DISTINCT business_key) ve bilinen toplamları karşılaştır. Sonucun doğru görünmesi, doğru olduğu anlamına gelmez.

8. JOIN Sonucunu Nasıl Doğrularsın?

1) Key uniqueness testi

```
SELECT customer_id, COUNT(*) AS n FROM customers GROUP BY customer_id HAVING COUNT(*) > 1;
```

PK olması beklenen kolon duplicate dönüyorsa veri modeli veya veri kalitesi problemi vardır.

2) Orphan record testi

```
SELECT o.order_id FROM orders o LEFT JOIN customers c ON c.customer_id = o.customer_id WHERE
c.customer_id IS NULL;
```

3) Satır sayısı karşılaştırması

```
SELECT COUNT(*) FROM orders; SELECT COUNT(*) FROM orders o JOIN customers c ON c.customer_id =
o.customer_id;
```

4) Finansal toplam reconciliation

Örneğin orders tablosundaki bilinen gelir toplamını, join sonrası elde ettiğin gelir toplamıyla karşılaştır. Beklenmeyen fark varsa duplicate, filtre veya grain sorunu araştır.

Mülakat cümlesi: “JOIN yazdıktan sonra cardinality varsayımımı kontrol eder, business key distinct sayısını ve temel KPI toplamlarını join öncesi/sonrası karşılaştırırım.” Bu, yalnızca syntax değil veri güvenilirliği düşündüğünü gösterir.

9. UNION ve UNION ALL

UNION benzer kolon yapısına sahip iki sorgu sonucunu satır bazında üst üste ekler.

UNION vs UNION ALL

Özellik	UNION	UNION ALL
Duplicate davranışı	Tekrarlı satırları kaldırır	Tüm satırları korur
Performans	Duplicate kontrolü nedeniyle daha maliyetlidir	Genellikle daha hızlıdır
Ne zaman?	Gerçekten tekilleştirme istendiğinde	Kaynakları olduğu gibi birleştirmek istendiğinde

```
SELECT order_id, order_date, total_amount, 'web' AS source FROM web_orders UNION ALL SELECT order_id, order_date, total_amount, 'store' AS source FROM store_orders;
```

Kolon uyumu: UNION edilen SELECT'lerin kolon sayısı aynı olmalı ve karşılıklı kolonların veri tipleri uyumlu olmalıdır. Kolon adları çoğunlukla ilk SELECT'ten gelir.

Analitik örnek

2025 ve 2026 arşiv tablolarını birleştirmek, web ve mağaza satışlarını tek akışa almak veya farklı kampanya sonuçlarını ortak rapor tablosunda toplamak.

10. Subquery: Sorgu İçinde Sorgu

Subquery başka bir SQL ifadesinin içinde çalışan alt sorgudur. Ara sonucu filtreleme, karşılaştırma veya sanal tablo olarak kullanma ihtiyacında işe yarar.

Scalar subquery

```
SELECT order_id, total_amount FROM orders WHERE total_amount > ( SELECT AVG(total_amount) FROM orders WHERE status = 'completed' );
```

IN ile subquery

```
SELECT customer_id, city FROM customers WHERE customer_id IN ( SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*) >= 5 );
```

FROM içinde derived table

```
SELECT customer_segment, AVG(customer_revenue) AS avg_revenue FROM ( SELECT customer_id, SUM(total_amount) AS customer_revenue, CASE WHEN SUM(total_amount) >= 10000 THEN 'High' ELSE 'Standard' END AS customer_segment FROM orders GROUP BY customer_id ) x GROUP BY customer_segment;
```

Okunabilirlik: Subquery büyüdükçe CTE kullanmak çoğu zaman daha okunabilir olur. CTE bu müfredatta bir sonraki “SQL İleri” modülünde ayrıntılı ele alınacak.

11. Gerçek Vaka: E-Ticaret Satış Analizi

Senaryo: Yönetim son çeyrekte gelirin yavaşladığını söylüyor. Hangi kategori, şehir ve müşteri segmentinin değişime katkı verdiğini anlaman isteniyor.

Adım 1 - Temiz satış tabanını kur

```
SELECT o.order_id, o.customer_id, o.order_date, c.city, p.category, oi.quantity, oi.unit_price, oi.quantity * oi.unit_price AS line_revenue FROM orders o JOIN customers c ON c.customer_id = o.customer_id JOIN order_items oi ON oi.order_id = o.order_id JOIN products p ON p.product_id = oi.product_id WHERE o.status = 'completed';
```

Adım 2 - Kategori × ay analizi

```
SELECT DATE_TRUNC('month', o.order_date) AS month, p.category, SUM(oi.quantity * oi.unit_price) AS revenue, COUNT(DISTINCT o.order_id) AS orders, COUNT(DISTINCT o.customer_id) AS customers FROM orders o JOIN order_items oi ON oi.order_id = o.order_id JOIN products p ON p.product_id = oi.product_id WHERE o.status = 'completed' GROUP BY 1, 2 ORDER BY 1, 3 DESC;
```

Adım 3 - Yorumu KPI bileşenlerine ayır

Gelir düşüşü tek başına neden değildir. Sipariş sayısı mı azaldı? Müşteri sayısı mı? Sepet tutarı mı? Kategori mix'i mi değişti? SQL sonucunu bu bileşenleri ayrıştıracak şekilde tasarla.

12. Vaka Devamı: Segment ve Root-Cause

Yüksek değerli müşterileri tanımla

```
SELECT customer_id, COUNT(*) AS completed_orders, SUM(total_amount) AS lifetime_revenue FROM orders WHERE status = 'completed' GROUP BY customer_id HAVING SUM(total_amount) >= 10000;
```

Şehir bazında sipariş ve AOV

```
SELECT c.city, COUNT(DISTINCT o.order_id) AS orders, SUM(o.total_amount) AS revenue, SUM(o.total_amount) / NULLIF(COUNT(DISTINCT o.order_id),0) AS aov FROM orders o JOIN customers c ON c.customer_id = o.customer_id WHERE o.status = 'completed' GROUP BY c.city HAVING COUNT(DISTINCT o.order_id) >= 20 ORDER BY revenue DESC;
```

Yöneticiye sunulabilecek sonuç yapısı

Bulgu	Kontrol sorusu	Olası aksiyon
Kategori geliri düştü	Sipariş mi AOV mi düştü?	Kategori kampanyası / stok kontrolü
Belirli şehirde sipariş azaldı	Trafik mi conversion mı?	Kanal ve operasyon analizi
High-value segment küçüldü	Tekrar sipariş oranı düştü mü?	Retention kampanyası
Gelir sabit, müşteri sayısı düştü	AOV artışı düşüşü maskeliyor mu?	Müşteri kazanımını incele

Analist çıktısı: SQL sonucu tek başına “insight” değildir. Insight, metrikteki değişimi iş bağlamında yorumlayıp hangi mekanizmanın değiştiğini göstermektir.

13. En Sık Yapılan SQL Orta Seviye Hataları

Hata	Neden tehlikeli?
GROUP BY grain’ini yanlış kurmak	Sipariş + ürün join’inde sipariş KPI’larını satır seviyesinde tekrar toplamak.
COUNT(*) ile müşteri saymak	Join sonrası aynı müşteri birden fazla satırda olabilir; COUNT(DISTINCT customer_id) gere
LEFT JOIN’i WHERE ile INNER’a çevirmek	Sağ tablo koşulunu WHERE’e koyup NULL satırları istemeden elemek.
Many-to-many join	İki detay tablosunu aynı parent key üzerinden join edip çarpan etkisi yaratmak.
UNION yerine UNION ALL / tersi	İstenmeyen tekilleştirme veya duplicate koruma.
AVG’yi yanlış grain’de almak	Sipariş ortalaması ile müşteri ortalamasını karıştırmak.
NULL’ı 0 sanmak	Eksik değer ile gerçek sıfır arasındaki iş farkını yok saymak.
Sonucu doğrulamamak	Sorgu çalıştı diye KPI’ı doğru kabul etmek.

Kural: Karmaşık bir sorgu yazmadan önce küçük parçaları ayrı ayrı çalıştır. Her join’den sonra satır sayısını ve distinct key sayısını kontrol et. Analitik SQL’de doğruluk, kısa sorgudan daha değerlidir.

14. Mülakat Soruları ve Beklenen Yaklaşım

Soru	Güçlü cevap özeti
WHERE ile HAVING farkı nedir?	WHERE satırları aggregate öncesi, HAVING grupları aggregate sonrası filtreler.
COUNT(*) ve COUNT(column) farkı?	COUNT(*) tüm satırları, COUNT(column) NULL olmayan değerleri sayar.
LEFT JOIN ne zaman kullanırsın?	Sol tablodaki tüm evreni korumak, eşleşmeyenleri de görmek istediğimde.
JOIN sonrası revenue iki katına çıktı. Ne kontrol edersin?	Cardinality, grain, duplicate key, many-to-many ilişki ve join anahtarları.
UNION ve UNION ALL farkı?	UNION duplicate kaldırır; UNION ALL korur ve genellikle daha az maliyetlidir.
Ortalama müşteri harcamasını nasıl hesaplarsın?	Önce müşteri bazında SUM, sonra bu müşteri toplamlarının AVG’si.
Hiç sipariş vermeyen müşterileri nasıl bulursın?	customers LEFT JOIN orders + WHERE order_id IS NULL.
Subquery ne zaman kullanılır?	Bir ara sonucu filtre, karşılaştırma veya derived table olarak kullanmak için.
Join doğruluğunu nasıl test edersin?	Satır sayıları, distinct business key, key uniqueness, orphan ve KPI reconciliation
GROUP BY sonucunun grain’ini nasıl açıklarsın?	GROUP BY’daki boyutların kombinasyonu sonuç satırının grain’ini belirler.

15. Uygulama Alıştırmaları

Aşağıdaki soruları önce kağıt üzerinde çözüm planına çevir, sonra SQL yaz. Her sorgunun grain’ini bir cümleyle belirt.

1. Her kategori için completed siparişlerden toplam gelir, sipariş sayısı ve tekil müşteri sayısını hesapla.

2. Aylık bazda en az 100 siparişi olan ayları HAVING ile getir.

3. Hiç sipariş vermeyen müşterileri listele.

4. En az 3 completed sipariş veren müşterilerin toplam harcamasını bul.

5. Her şehir için AOV hesapla; 10’dan az siparişi olan şehirleri çıkar.

6. Web ve mağaza sipariş tablolarını UNION ALL ile birleştir ve source alanı ekle.

7. Ortalama completed sipariş tutarının üzerinde olan siparişleri subquery ile getir.

8. customers tablosunda duplicate customer_id olup olmadığını test et.

9. orders içinde customers tablosunda karşılığı olmayan customer_id’leri bul.

10. order_items ve payments join’inde double-counting yaratabilecek bir senaryo oluştur ve düzelt.

11. Kategori bazında quantity ve revenue hesapla, revenue > 50.000 olan kategorileri göster.

12. Müşteri bazında lifetime revenue hesaplayıp 10.000+ olanları “High Value” olarak seç.

Çalışma standardı: Her alıştırmada 1) business question, 2) grain, 3) gerekli tablolar, 4) join key, 5) filtre, 6) aggregate, 7) doğrulama kontrolü başlıklarını yaz.

16. Modül Sonu Projesi: SQL Sales Analysis Case

Senaryo

Bir e-ticaret şirketinin customers, orders, order_items, products, payments tabloları veriliyor. Yönetim son 6 ay performansını değerlendirmek istiyor.

Teslim etmen gereken analizler

- Aylık revenue, order count, unique customer ve AOV.
- Kategori ve şehir bazında satış performansı.
- En yüksek gelir yaratan müşteri segmenti.
- Hiç sipariş vermeyen kayıtlı müşteriler.
- En az 5 sipariş veren sadık müşteriler.
- Payment toplamı ile satış toplamı arasında reconciliation.
- JOIN kaynaklı duplicate risklerinin kontrolü.
- En az 5 maddelik executive insight ve öneri.

GitHub / portfolio çıktısı

Dosya	İçerik
schema.md	Tablolar, grain, PK/FK ve ilişki açıklaması
analysis.sql	Temiz, yorumlu SQL sorguları
validation.sql	Duplicate, orphan, reconciliation ve satır sayısı kontrolleri
README.md	İş problemi, yaklaşım, bulgular, öneriler

Portfolyo hedefi: Sadece “SQL biliyorum” demek yerine, veri modelini anlayıp güvenilir KPI üretebildiğini gösteren bir analiz dosyası oluştur.

17. SQL Orta - Hızlı Referans

İhtiyaç	Kalıp
Grup bazında toplam	SELECT dim, SUM(x) FROM t GROUP BY dim
Gruplanmış sonucu filtrele	... GROUP BY dim HAVING SUM(x) > n
Tekil müşteri	COUNT(DISTINCT customer_id)
Eşleşen kayıtlar	A INNER JOIN B ON A.key = B.key
Sol evreni koru	A LEFT JOIN B ON A.key = B.key
Eşleşmeyeni bul	LEFT JOIN ... WHERE B.key IS NULL
Satırları üst üste ekle	SELECT ... UNION ALL SELECT ...
Ortalamadan büyük kayıtlar	WHERE x > (SELECT AVG(x) FROM t)
Duplicate key testi	GROUP BY key HAVING COUNT(*) > 1
Join sonrası güvenlik	COUNT(*), COUNT(DISTINCT key), KPI reconciliation

Modül tamamlandıktan sonra yapabiliyor olmalısın

- GROUP BY ile iş KPI'larını doğru grain'de üretmek.
- WHERE ve HAVING'i bilinçli kullanmak.
- INNER ve LEFT JOIN davranışlarını açıklamak.
- Join cardinality ve double-counting riskini tespit etmek.
- UNION / UNION ALL seçimini gerekçelendirmek.
- Subquery ile ara sonuç kullanmak.
- SQL sonucunu veri kalite kontrolleriyle doğrulamak.

Sıradaki modül: SQL İleri - CTE, Window Functions, ROW_NUMBER, RANK, LAG/LEAD, date functions, cohort ve funnel sorguları.